Introduction to regular expressions

REGULAR EXPRESSIONS IN PYTHON

What is a regular expression?

> **REGular EXpression or regex:**
>
> *String containing a combination of normal characters and special metacharacters that describes patterns to find text or positions within a text*

r'st\d\s\w{3,10}'

Regular expressions

- Regular expressions (a.k.a. RE, regexp, regexes, regex) are a highly specialized **text-matching tool.**

- Regex can be viewed as a tiny programming language embedded in Python and made available through the **re** module.

- They are extremely useful in searching and modifying (long) string

- *http://docs.python.org/library/re.html*

Not only in Python

- REs are very widespread:
  - Unix utility “grep”
  - Perl
  - TextWrangler
  - TextPad
  - Python

- So, … learning the “RE language” would serve you in many different environments as well.

What is a regular expression?

> **REGular EXpression or regex:**
>
> *String containing a combination of normal characters and special metacharacters that describes patterns* ***to find text or positions within a text***

- Pattern matching usage:
  - Find and replace text
  - Validate strings
- Very powerful and fast

Finding a good match

- **Using this RE tiny language, you can specify <u>patterns</u> that you want to <u>match</u>**

- You can then ask ***match*** questions such as:
  - "Does this string match this pattern?"
  - "Is there a match to this pattern anywhere in this string?"
  - "What are all the matches to this pattern in this string?"

- You can also use REs to **modify** a string
  - **Replace** parts of a string (sub) that match the pattern with something else
  - Break stings into smaller pieces (split) wherever this pattern is matched

A simple example

- Consider the following example:

```
>>> import re
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')

['foot', 'fell', 'fastest']
```

This RE means: A word that starts with 'f' followed by any number of alphabetical characters

- Note the *re.* prefix – findall is a function in the re module
- findall:
  - Format: `findall(<regexe>, <string>)`
  - Returns a list of all non-overlapping substrings that matches the regexe.
- REs are provided as strings.

```python
import re
```

- Find all matches of a pattern:

re.findall(r"regex", string)

```python
re.findall(r"#movies", "Love #movies! I had fun yesterday going to the #movies")
```

```
['#movies', '#movies']
```

Remember:
It's all about matching

Regular expressions are patterns;
they "match" sequences of characters

```python
import re
```

- Split string at each match:

re.split(r"regex", string)

```python
re.split(r"!", "Nice Place to eat! I'll come back! Excellent meat!")
```

```
['Nice Place to eat', " I'll come back", ' Excellent meat', '']
```

```python
import re
```

- Replace one or many matches with a string:

re.sub(r"regex", new, string)

```python
re.sub(r"yellow", "nice", "I have a yellow car and a yellow house in a yellow neighborhood")
```

```
'I have a nice car and a nice house in a nice neighborhood'
```

```python
import re

text = "apple,banana,orange,grape"
pattern = r","  # Comma is the delimiter

result = re.split(pattern, text)

print(result)
```

Output

```
['apple', 'banana', 'orange', 'grape']
>>>
```

```python
import re

text = "Hello, World!"
pattern = r"World"
replacement = "Python"

result = re.sub(pattern, replacement, text)

print(result)
```

Output

```
Hello, Python!
>>>
```

Supported metacharacters

Metacharacter	Meaning
\d	Digit

```
re.findall(r"User\d", "The winners are: User9, UserN, User8")
```

```
['User9', 'User8']
```

Metacharacter	Meaning
\D	Non-digit

```
re.findall(r"User\D", "The winners are: User9, UserN, User8")
```

```
['UserN']
```

Supported metacharacters

Metacharacter	Meaning
\w	Word

Metacharacter	Meaning
\W	Non-word

```
re.findall(r"User\w", "The winners are: User9, UserN, User8")
```

```
['User9', 'UserN', 'User8']
```

```
re.findall(r"\W\d", "This skirt is on sale, only $5 today!")
```

```
['$5']
```

Supported metacharacters

Metacharacter	Meaning
\s	Whitespace

Metacharacter	Meaning
\S	Non-Whitespace

```
re.findall(r"Data\sScience", "I enjoy learning Data Science")
```

```
['Data Science']
```

```
re.sub(r"ice\Scream", "ice cream", "I really like ice-cream")
```

```
'I really like ice cream'
```

Basic RE matching

- Most letters and numbers match themselves
  - For example, the regular expression `test` will match the string **`test`** exactly
  - Normally case sensitive

```
>>> re.findall(r'test', "Tests are testers' best testimonials")
['test', 'test']
```

Sets

- **Square brackets** mean that any of the listed characters will do (matching one of several alternatives)
  - `[abc]` means either ”a” , ”b” , or “c”
- You can also give a range:
  - `[a-d]` means ”a”, ”b”, ”c”, or ”d”
- Negation: caret means ***not***
  - `[^a-d]` means anything but a, b, c or d
  - `[^5]` means anything but 5
- Metacharacters are not active inside sets.
  - `[ak$]` will match “a”, “k”, or “$”. Normally, “$” is a metacharacter. Inside a set it’s stripped of its special nature.

Predefined sets

- `\d` matches any decimal digit (equivalent to `[0-9]`).
- `\D` matches any non-digit character (equivalent to `[^0-9]`).
- `\s` matches any whitespace character (equivalent to `[ \t\n\r\f\v]`).
- `\S` matches any non-whitespace character (equivalent to `[^ \t\n\r\f\v]`).
- `\w` matches any alphanumeric character (equivalent to `[a-zA-Z0-9_]`).
- `\W` matches any non-alphanumeric character (equivalent to the class `[^a-zA-Z0-9_]`.

Note the pairs.
Easy to remember!

Matching boundaries

- `^` matches the beginning of the string
- `$` matches the end of the string

- `\b` matches a word boundary
- `\B` matches position that is not a word boundary

(A word boundary is a position that changes from a word character to a non-word character, or vice versa).

For example, `\bcat` will match **`catalyst`** but not **`location`**

Wildcards

- . matches **any** character (except newline)
- If you really mean "." you must use a backslash

- WARNING:
  - backslash is special in Python strings
  - It's special again in RE
  - This means you need too many backslashes
  - Use "raw strings" to make things simpler

- What does this RE means: `r'\d\.\d'`?

- What does this RE means: `r'\d\.\d'`?

```python
import re

text = "The value is 3.5"
pattern = r'\d\.\d'

matches = re.findall(pattern, text)

print(matches)
```

output

```
['3.5']
>>>
```

```python
import re


# Pattern to match any character followed by "an"
matches = re.findall(r'.an', "apple banana cherry canoano")

print(matches)
```

output

```
['ban', 'can', 'oan']
>>
```

- This will work:

```
>>> import re
>>> mystring = "This contains 2 files, hw3.py and uppercase.py."
>>> all_matches = re.findall(r' [^ ]+\.py', mystring)
>>> print all_matches
```

```
['hw3.py','uppercase.py']
```

```
r".+\.py"   "Two files: hw3.py and upper.py."
r"\w+\.py"  "Two files: hw3.py and UPPER.py."
```

Repetitions

- Allows you to specify that a portion of the RE must/can be repeated a certain number of times.
- **`*`** : The previous character can repeat 0 or more times
  - `ca*t` matches "ct", "cat", "caat", "caaat" etc.
- **`+`** : The previous character can repeat 1 or more times
  - `ca+t` matches "cat", "caat" etc. but not "ct"
- Braces provide a more detailed way to indicate repeats
  - `A{1,3}` means at least one and no more than three A's
  - `A{4,4}` means exactly four A's

Repetitions

REGULAR EXPRESSIONS IN PYTHON

Repeated characters

Validate the following string:

password1234

Repeated characters

Validate the following string:

password1234

Repeated characters

Validate the following string:

password1234

Repeated characters

Validate the following string:

password1234

```python
import re
password = "password1234"
```

```python
re.search(r"\w\w\w\w\w\w\w\w\d\d\d\d", password)
```

```
<_sre.SRE_Match object; span=(0, 12), match='password1234'>
```

Repeated characters

Validate the following string:

password1234

```
import re
password = "password1234"
```

```
re.search(r"\w{8}\d{4}", password)
```

```
<_sre.SRE_Match object; span=(0, 12), match='password1234'>
```

> **Quantifiers**:
>
> A metacharacter that tells the regex engine how many times to match a character immediately to its left.

```
text = "Date of start: 4-3. Date of registration: 10-04."
print(re.findall(r" ", text))
```

Output:

[' ', ' ', ' ', ' ', ' ', ' ', ' ']

```
text = "Date of start: 4-3. Date of registration: 10-04."
print(re.findall(r“\d+- ", text))
```

Output:

['4-', '10-']

```
text = "Date of start: 4-3. Date of registration: 10-04."
print(re.findall(r“\d+-\d+ ", text))
```

Output:

['4-3', '10-04']

Quantifiers

- Once or more: `+`

Quantifiers

- Zero times or more: `*`

```
my_string = "The concert was amazing! @ameli!a @joh&&n @mary90"
re.findall(r"@\w+\W*\w+", my_string)
```

```
['@ameli!a', '@joh&&n', '@mary90']
```

Quantifiers

- Zero times or once: `?`

```
text = "The color of this image is amazing. However, the colour blue could be brighter."
re.findall(r"colou?r", text)
```

```
['color', 'colour']
```

Quantifiers

- n times at least, m times at most : `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
```

```
re.findall(r"                          ", phone_number)
```

Quantifiers

- n times at least, m times at most : `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
```

```
re.findall(r"\d{1,2}-                    ", phone_number)
```

Quantifiers

- n times at least, m times at most : `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
```

```
re.findall(r"\d{1,2}-\d{3}-          ", phone_number)
```

Quantifiers

- n times at least, m times at most : `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
```

```
re.findall(r"\d{1,2}-\d{3}-\d{2,3}-\d{4,}", phone_number)
```

```
['1-966-847-3131', '54-908-42-42424']
```

The MATCH Method

```
import re

pattern = '^a...s$'
test_string = 'abyss'
result = re.match(pattern, test_string)

if result:
  print("Search successful.")
else:
    print("Search
  unsuccessful.")
```

The pattern is: **any five letter string starting with a and ending with s**.

OUTPOUT:

Search successful.

Here, we used re.match() function to search pattern within the test_string. The method returns a match object if the search is successful. If not, it returns None.

The SEARCH Method

The re.search() method takes two arguments: a pattern and a string. The method looks
for the first location where the RegEx pattern produces a match with the string.

If the search is successful, re.search() returns a match object; if not, it returns None.

```
match = re.search(pattern, str)
```

```python
import re

string = "Python is fun"

# check if 'Python' is at the beginning
match = re.search('\APython', string)

if match:
  print("pattern found inside the string")
else:
  print("pattern not found")

# Output: pattern found inside the string
```

Example : Match object

```
import re

string = '39801 356, 2102 1111'

# Three digit number followed by space followed by two digit number
pattern = '(\d{3}) (\d{2})'

# match variable contains a Match object.
match = re.search(pattern, string)

if match:
  print(match.group())
else:
  print("pattern not found")

# Output: 801 35
```

Here, `match` variable contains a match object.

Our pattern `(\d{3}) (\d{2})` has two subgroups `(\d{3})` and `(\d{2})`. You can get the part of the string of these parenthesized subgroups. Here's how:

```
>>> match.group(1)
'801'

>>> match.group(2)
'35'
>>> match.group(1, 2)
('801', '35')

>>> match.groups()
('801', '35')
```

match.start(), match.end() and match.span()

The `start()` function returns the index of the start of the matched substring. Similarly, `end()` returns the end index of the matched substring.

```
>>> match.start()
2
>>> match.end()
8
```

The `span()` function returns a tuple containing start and end index of the matched part.

```
>>> match.span()
(2, 8)
```

Looking for patterns

Two different operations to find a match:

re.search(r"regex", string)

```
re.search(r"\d{4}", "4506 people attend the show
```

```
<_sre.SRE_Match object; span=(0, 4), match='4506
```

```
re.search(r"\d+", "Yesterday, I saw 3 shows")
```

```
<_sre.SRE_Match object; span=(17, 18), match='3'
```

re.match(r"regex", string)

```
re.match(r"\d{4}", "4506 people attend the show"
```

```
<_sre.SRE_Match object; span=(0, 4), match='4506
```

```
print(re.match(r"\d+","Yesterday, I saw 3 shows"
```

```
None
```

Example

```python
import re

# Target String
target_string = "Emma is a baseball player who was born on June 17, 1993."

# find substring 'ball'
result = re.search(r"ball", target_string)

# Print matching substring
print(result.group())
# output 'ball'

# find exact word/substring surrounded by word boundary
result = re.search(r"\bball\b", target_string)
if result:
    print(result)
# output None

# find word 'player'
result = re.search(r"\bplayer\b", target_string)
print(result.group())
# output 'player'
```

Example to search multiple patterns

```
import re

target_string = "Emma is a basketball player who was born on June 17."

# two group enclosed in separate ( and ) bracket
result = re.search(r"(\w{10}).+(\d{2})", target_string)

# Extract the matches using group()

# print ten-letter word
print(result.group(1))
# Output basketball

# print two digit number
print(result.group(2))
# Output 17
```

re.match(pattern, string, flags=0)

Try to apply the pattern at the start of the string, returning a Match object, or None if no match was found

Example

1. **Match regex pattern at the beginning of the string**

```
res = re.match(r"\b\w{5}\b", "Jessa and Kelly")
```

Target string

Regex pattern in string format (Look for 5-letter word)

Result: Jessa

2. **Match regex pattern anywhere in the string and get only the first match**

```
res = re.search(r"\b\w{5}\b",  "Jessa and Kelly")
```

Result: Jessa

3. **Find all the matches to a regex pattern.**

```
res = re.findall(r"\b\w{5}\b", "Jessa and Kelly")
```

Result: ["Jessa", "Kelly"]

Example to match six-letter word anywhere in the string

```python
import re

target_string = "Jessa loves Python and pandas"
# Match six-letter word
pattern = r"\w{6}"

# match() method
result = re.match(pattern, target_string)
print(result)
# Output None

# search() method
result = re.search(pattern, target_string)
print(result.group())
# Output 'Python'

# findall() method
result = re.findall(pattern, target_string)
print(result)
# Output ['Python', 'pandas']
```

The re.Match Object

Contains the information about matching strings

PYnative.com

```
match_obj = re.match(r"\b\w{5}\b", "Jessa and Kelly")
Print(match_obj )
```

<re.Match object; span=(0, 5), match='Jessa'>

Now query the match object to get info about the matching string

1. match_obj().**group()** → Jessa
2. match_obj.**start()** → 0
3. match_obj.**end()** → 5
4. match_obj.**span()** → (0, 5)

```
import re
target_string = "Jessa and
Kelly" # Match five-letter word
res = re.match(r"\b\w{5}\b",
target_string) # printing entire match
object
print(res)
# Output re.Match object; span=(0, 5),
match='Jessa' # Extract Matching value
print(res.group())
# Output Jessa
# Start index of a
match print(res.start())
# Output 0
# End index of a
match
print("End index: ", res.end()) # Output
5 # Start and end index of a match
pos =
res.span()
print(pos)
# Output (0, 5)
# Use span to
retrieve the
```

Search vs. match

Python RE module offers two different methods to perform regex pattern matching.

- The ***match()*** checks for a match only at the beginning of the
- string.

How re.match() works

The match method returns a corresponding match object instance if zero or more characters at the beginning of the string match the regular expression pattern.

In simple words, the `re.match` returns a match object only if the pattern is located at the beginning of the string; otherwise, it will return None.

How re.search() works

On the other hand, the search method scans the entire string to look for a pattern and returns only the first match. I.e., As soon as it gets the first match, it stops its execution.

Let's see the example to understand the difference between search and match. In this example, we will see how to match the regex pattern using the match and search method.

```python
import re

target_string = "Emma is a baseball player who was born on June 17, 1993"

# Match 2-digit number
# Using match()
result = re.match(r'\d{2}', target_string)
print(result)
# Output None

# Using search()
result = re.search(r'\d{2}', target_string)
print(result.group())
# Output 17
```

More examples

```
>>> re.sub('\d', 'x', 'a_b - 12')
'a_b - xx'
>>> re.sub('\D', 'x', 'a_b - 12')
'xxxxxx12'
>>> re.sub('\s', 'x', 'a_b - 12')
'a_bx-x12'
>>> re.sub('\S', 'x', 'a_b - 12')
'xxx x xx'
>>> re.sub('\w', 'x', 'a_b - 12')
'xxx - xx'
>>> re.sub('\W', 'x', 'a_b - 12')
'a_bxxx12`
>>> re.sub('^', 'x', 'a_b - 12')
'xa_b - 12'
>>> re.sub('$', 'x', 'a_b - 12')
'a_b - 12x'
>>> re.sub('\b', 'x', 'a_b - 12')
'a_b - 12'
>>> re.sub('\\b', 'x', 'a_b - 12')
'xa_bx - x12x'
>>> re.sub(r'\b', 'x', 'a_b - 12')
'xa_bx - x12x'
>>> re.sub('\B', 'x', 'a_b - 12')
'ax_xb x-x 1x2'
```